

आधुनिक कृत्रिम बुद्धिमत्ता

BIBLIA

AI का आधुनिक उपयोग, subagents, SDD, AGENTS.md,
GitHub, prompt engineering और harness engineering.

लेखकत्व

Nicolás Ezequiel Melluso

nicolas.e.melluso@gmail.com

linkedin.com/in/nicolas-ezequiel-melluso

github.com/Nicolas-Melluso

सामान्य अनुक्रमणिका

- 01** **कृत्रिम बुद्धिमत्ता का आधुनिक उपयोग**
मॉडल से चैट करने से लेकर सोच, निष्पादन और सत्यापन की प्रणाली के साथ काम करने तक

- 02** **सबएजेंट्स का बुद्धिमान उपयोग**
बेहतर डेलीवेशन के लिए मानदंड, पैटर्न और ऑपरेशनल क्लोज़र

- 03** **SDD और सपोर्ट संरचना**
AI-सक्षम repositories में लागू Specification-Driven Development

- 04** **AGENTS.md, .github और Prompt-स्टाइल Commands**
agents, SDD, Copilot, workflows और reusable prompts के लिए तैयार repository कैसे बनाएं

- 05** **Prompt Engineering और Harness Engineering**
अलग-अलग prompts से versioned, evaluable और production-ready systems तक

खंड 01

कृत्रिम बुद्धिमत्ता का आधुनिक उपयोग

मॉडल से चैट करने से लेकर सोच, निष्पादन और सत्यापन की प्रणाली के साथ काम करने तक

केंद्रीय विचार

AI का आधुनिक उपयोग यह नहीं है कि चैट खोलो, "यह कर दो" कहो और पहली प्रतिक्रिया मान लो। वह पहला चरण था। आधुनिक चरण यह है कि AI को काम की एक परत की तरह माना जाए: सहायक, तकनीकी साथी, शोधकर्ता, निष्पादक, समीक्षक और मेमोरी सिस्टम का संयोजन। फर्क लंबे prompt लिखने में नहीं है। फर्क उस फ्लो को डिजाइन करने में है जहाँ हर इंटरैक्शन पुनः उपयोग योग्य context, artifacts, tests और decisions छोड़कर जाए।

AI का अपरिपक्व उपयोग बातचीत-आधारित और disposable होता है: कुछ पूछा, जवाब मिला, थोड़ा कॉपी किया और आगे बढ़ गए। परिपक्व उपयोग operational होता है: उद्देश्य तय करो, context दो, constraints स्पष्ट करो, काम को हिस्सों में बाँटो, परिणाम सत्यापित करो और जो सीखा उसे दर्ज करो। मूल्य तब आता है जब AI सिर्फ टेक्स्ट मशीन न रहकर development, research या production प्रक्रिया का विस्तार बन जाता है।

यह खंड एक सरल कार्य-मॉडल प्रस्तावित करता है: AI को चार परतों वाली प्रणाली की तरह सोचो।

1. Context layer: कार्रवाई से पहले AI को क्या जानना चाहिए।
2. Task layer: अभी उसे क्या उत्पन्न करना है।
3. Verification layer: कैसे पता चले कि परिणाम काम का है।
4. Memory layer: कहाँ दर्ज रहे ताकि वही reasoning फिर न दोहरानी पड़े।

जब ये चारों परतें मौजूद हों, AI गंभीर कामों में मदद कर सकता है: specifications लिखना, code review करना, विकल्पों की तुलना करना, documentation बनाना, tests चलाना, risks पहचानना, presentations तैयार करना, किसी व्यक्ति को train करना या technical decisions तेज करना। जब ये न हों, AI कभी-कभी बहुत प्रभावशाली और चुपचाप खतरनाक हो जाता है।

व्यवहार में क्या बदला

महत्वपूर्ण बदलाव सिर्फ मॉडल के बेहतर होने का नहीं है। बदलाव यह है कि अब हम ऐसे agents के साथ काम कर सकते हैं जो tools से जुड़े हों: editor, terminal, browser, repository, issue tracker, documentation, local databases, tests, linters, emulators और pipelines। पहले मॉडल काम के बाहर से जवाब देता था। अब वह काम के भीतर भाग ले सकता है।

इससे मांग रखने का तरीका बदलना पड़ता है। आधुनिक अनुरोध सिर्फ "X समझाओ" नहीं कहता। वह कहता है: "इन files को पढ़ो, current behavior पहचानो, सीमित बदलाव प्रस्तावित करो, उसे implement करो, ये tests चलाओ और risks सहित summary दो।" AI अब oracle नहीं, contract के तहत operator बन जाता है।

मानव भूमिका भी बदलती है। अब जीत इस बात से नहीं मिलती कि किसने सब कुछ हाथ से टाइप किया। जीत अच्छे context, अच्छे constraints, अच्छे acceptance criteria और अच्छे verification mechanisms तय करने से मिलती है। AI बहुत कुछ बना सकता है, पर वह अपने आप नहीं जानता कि आपके business के लिए कौन-सा tradeoff सही है, कौन-सा legal risk स्वीकार्य है, कौन-सा technical debt सहना है, या आप कौन-सा user experience बचाना चाहते हैं।

आधुनिक प्रश्न यह नहीं है "कौन-सा prompt उपयोग करें।" आधुनिक प्रश्न है "ऐसी कौन-सी कार्य-प्रणाली है जो हर हफ्ते परिणाम बेहतर बनाती है।"

अनुशंसित कार्य-चक्र

AI के साथ एक मजबूत फ्लो ऐसा दिख सकता है:

Intent -> Context -> Plan -> Execution -> Verification -> Record -> Next iteration

Intent इच्छित परिणाम तय करता है। Context अस्पष्टता घटाता है। Plan आवेगपूर्ण काम से बचाता है। Execution ठोस artifacts देता है। Verification विश्वसनीय दिखने वाली और वास्तव में सही चीज़ में अंतर करता है। Record चैट में ज्ञान खोने से बचाता है। अगला iteration काम को संचित सीख में बदल देता है।

एक सरल उदाहरण:

```
Intent:
  मैं magic link authentication जोड़ना चाहता हूँ।

Context:
  Node/TypeScript repo, PostgreSQL, service architecture, Vitest tests।

Plan:
  1. मौजूदा auth ढूँढो।
  2. token table जोड़ो।
  3. service implement करो।
  4. unit और integration tests जोड़ो।
  5. environment variables document करो।

Verification:
  npm test
  npm run typecheck
  login -> link -> session फ्लो का manual test

Record:
  छोटा ADR: password के बजाय magic link क्यों।
  expected behavior की spec।
  rollback checklist।
```

यह फ्लो किसी एक खास tool पर निर्भर नहीं है। यह Codex, Copilot, Cursor, Claude Code, Gemini CLI या custom agent के साथ काम करता है। परिपक्वता प्रक्रिया में है।

Context की न्यूनतम इकाई

AI तब बेहतर काम करता है जब उसे पैकेज्ड context मिले, बिखरी हुई जानकारी नहीं। गंभीर कार्य के लिए context की न्यूनतम इकाई में यह शामिल होना चाहिए:

तत्व	किसलिए	उदाहरण
उद्देश्य	गलत चीज़ optimize होने से रोकता है	"abandoned checkout errors कम करना"
वर्तमान स्थिति	शुरुआती बिंदु देता है	"Stripe return <code>/checkout/success</code> पर आता है"
constraints	समाधानों को सीमित करता है	"prices न छेड़ें, provider migrate न करें"
relevant files	blind exploration कम करता है	<code>src/server.js</code> , <code>public/app.js</code>
acceptance criteria	closure परिभाषित करता है	"payment से लौटे तो purchased state दिखे"
verification	testing अनिवार्य करता है	"local smoke test और unit test"
risks	नाजुक हिस्सों को visible करता है	"logs में secrets leak न हों"

इस न्यूनतम इकाई के बिना मॉडल assumptions से खाली जगह भरता है। कभी सही होता है। वास्तविक systems में कभी-कभी वह ऐसी logic पर चलता है जो उचित लगती है पर प्रोजेक्ट की नहीं होती, और चीजें तोड़ देता है।

AI के लिए अच्छे कार्य

AI खास तौर पर तब मजबूत है जब वह explicit जानकारी पर काम कर सके और output सत्यापित हो सके। कुछ high-value उपयोग:

1. मौजूदा code का सार और map बनाना।
2. किसी idea को reviewable specification में बदलना।
3. acceptance criteria से test cases बनाना।
4. docs, code और tests में inconsistencies ढूँढना।
5. test suite के साथ सीमित क्षेत्र का refactor करना।
6. migration या validation scripts तैयार करना।
7. repeatable operations के लिए runbooks बनाना।
8. concrete rules के साथ PRs review करना।
9. conversations को actionable tasks में बदलना।
10. किसी और व्यक्ति के लिए training material बनाना।

जब AI से बिना data के निर्णय, business policies invent करना, एक साथ system के कई हिस्सों को छूना, permissions के बिना infrastructure बदलना, या human review के बिना "final" content बनवाना हो, तब इसकी विश्वसनीयता घटती है। इसका मतलब यह नहीं कि वह मदद नहीं कर सकता। मतलब यह है कि मजबूत control frame चाहिए।

आधुनिक prompt

आधुनिक prompt एक operational brief जैसा होता है। उसे काव्यात्मक या बहुत बड़ा होने की जरूरत नहीं। उसे ambiguity हटानी चाहिए।

Objective:

मैं चाहता हूँ कि तुम इस idea को implementation-ready SDD specification में बदलो।

Context:

प्रोडक्ट claims manage करने वाली एक B2B app है। repo Node.js, TypeScript, PostgreSQL और GitHub Actions उपयोग करता है। हम scope छोटा रखना चाहते हैं।

Input:

Idea: "एक operator को claim दूसरे team को reassign करने देना।"

Constraints:

- अभी पूरी screen design मत करो।
- जरूरी न हो तो नए roles assume मत करो।
- business rules को UI से अलग रखो।
- risks और open questions शामिल करो।

Output:

1. समस्या का सारांश।
2. Functional requirements।
3. Non-functional requirements।
4. Acceptance criteria।
5. Edge cases।
6. 3 slices में implementation plan।
7. Recommended tests।

Quality:

अगर जानकारी कम है तो उसे open question के रूप में चिह्नित करो। business data invent मत करो।

यह format तीन काम करता है: दिशा देता है, सीमाएँ तय करता है और response का evaluation कैसे होगा यह परिभाषित करता है। इसका उद्देश्य मॉडल को "inspire" करना नहीं, बल्कि उसे task के लिए contract करना है।

चैट से repository तक

सबसे महत्वपूर्ण छलांग है ज्ञान को चैट से repository में ले जाना। चैट नाजुक होती है: खो जाती है, अपने आप से टकराती है, versioning खराब होती है, PR में review नहीं होती, और CI में नहीं चलती। repository इसके उलट instructions, specs, decisions, prompts, tests और workflows सुरक्षित रख सकती है।

आधुनिक संगठन आमतौर पर यह विभाजन रखते हैं:

Artifact	Audience	Function
README.md	नए humans	प्रोजेक्ट और शुरुआत का तरीका बताना
AGENTS.md	code agents	operational rules, commands, style और verification
.github/copilot-instructions.md	Copilot	repo responses के लिए general instructions
.github/instructions/*.instructions.md	path-wise Copilot	code के हिस्सों के लिए specific rules
.github/prompts/*.prompt.md	humans और assistants	reusable prompt-style commands
.github/orquestador/sdd/*	टीम और agents	specs, decisions, tasks, traceability
.github/workflows/*	CI/CD	executable automation

व्यावहारिक नियम: जो दोहराया जाता है, उसे file बनना चाहिए। अगर हर बार मदद मांगने पर वही commands, वही style, वही constraints और वही test criteria फिर समझाने पड़ें, तो वह prompt engineering नहीं, context debt है।

छोटी टीम में AI की भूमिकाएँ

भले ही एक tool "assistant" जैसा दिखे, roles में सोचना उपयोगी है:

Role	क्या करता है	अच्छा output
Researcher	पढ़ता, तुलना करता, सारांश बनाता, patterns खोजता	file map, risks, questions
Planner	काम को विभाजित करता	slices वाला plan, dependencies, verification
Implementer	files बदलता	scoped patch, tests, summary
Reviewer	errors ढूँढता	file और line के साथ findings
Documenter	काम को knowledge में बदलता	README, ADR, runbook
Evaluator	behavior test करता	commands और results की report

subagents इस separation को औपचारिक बनाते हैं, पर मानसिक मॉडल एक चैट में भी काम करता है। जब एक ही बातचीत सब कुछ एक साथ करने लगती है, वह भ्रमित हो जाती है। जब हर role की concrete output हो, काम controllable बनता है।

Verification वैकल्पिक नहीं है

AI convincing text और सही दिखने वाला code बना सकता है। गंभीर रक्षा सिर्फ verification है। verification automated हो सकती है या human, लेकिन होनी चाहिए।

verification के उदाहरण:

- 1. Unit और integration tests।

2. Typecheck, lint और build।
3. Documented manual smoke test।
4. Acceptance criteria के विरुद्ध तुलना।
5. File-by-file diff review।
6. Real data या fixtures के साथ test।
7. prompts के लिए golden cases से evaluation।
8. Security और permissions checklist।

"अच्छा लग रहा है" पर्याप्त नहीं है। आधुनिक फ्लो मांगता है कि AI बताए: उसने क्या चलाया, क्या नहीं चला सका, क्या बदला, क्या बाकी है और क्या risk शेष है।

उपयोगी मेमोरी, अनंत मेमोरी नहीं

मेमोरी तब उपयोगी है जब वह दोहराव घटाए और consistency बढ़ाए। जब वह लंबे नोट्स का कूड़ाघर बन जाए, तब उपयोगी नहीं रहती। उपयोगी मेमोरी की तीन विशेषताएँ हैं:

1. Retrievable: ज्ञात path में हो।
2. Actionable: decisions, commands, conventions या सीखी गई गलतियाँ शामिल हों।
3. Verifiable: जब repo की current state बदल सकती हो, तब यह उसे replace न करे।

अच्छी मेमोरी के उदाहरण:

- repo context folder के रूप में ``.github/orquestador`` उपयोग करता है।
- workflows ही executable layer हैं; catalog सिर्फ documentation है।
- runtime changes बंद करने से पहले ``npm test`` और ``npm run build`` चलाओ।
- Windows पर folders rename करने से पहले locks verify करो।

खराब मेमोरी के उदाहरण:

- प्रोजेक्ट महत्वपूर्ण है।
- कभी-कभी fail होता है।
- good practices उपयोग करो।

आधुनिक मेमोरी भावनाएँ नहीं, operations सहेजती है।

30-दिन की adoption योजना

सप्ताह 1: context व्यवस्थित करें

`AGENTS.md` बनाओ, वास्तविक commands लिखो, constraints सूचीबद्ध करो और तय करो SDD कहाँ रहेगा। उद्देश्य सब कुछ कवर करना नहीं है। उद्देश्य यह है कि कोई agent repo में आए और आधा घंटा अंदाज़ा लगाने में न गंवाए।

Deliverables:

1. प्रारंभिक `AGENTS.md` |
2. अपडेटेड `README.md` |
3. `.github/orquestador/context/product.md` |
4. `.github/orquestador/context/architecture.md` |
5. verification commands की सूची।

सप्ताह 2: specifications के आधार पर काम

एक छोटी feature चुनो और implement करने से पहले spec लिखो। acceptance criteria, edge cases, non-goals और tests शामिल करो। फिर AI से सिर्फ एक slice implement करवाओ।

Deliverables:

1. पहली SDD spec |
2. slices वाला plan |
3. relevant technical decision हो तो ADR |
4. संबंधित tests |

सप्ताह 3: reusable prompts लाएँ

दोहराए जाने वाले कार्यों के लिए prompts बनाओ: feature planning, PR review, test generation, ADR writing, runbook preparation। इन्हें `.github/prompts` या चुने गए orchestration folder में रखो।

Deliverables:

1. `plan-feature.prompt.md` |
2. `review-pr.prompt.md` |
3. `write-adr.prompt.md` |
4. `generate-tests.prompt.md` |

सप्ताह 4: गुणवत्ता मापें

सरल harnesses या evaluations जोड़ो। बड़ी platform खड़ी करने की जरूरत नहीं। fixtures, expected cases और output compare करने वाले script से शुरुआत करो।

Deliverables:

1. `evals/` folder |
2. Input fixtures |
3. Evaluation rubric |
4. Local script या validation workflow |

सामान्य anti-patterns

Anti-pattern	लक्षण	सुधार
हर चीज़ के लिए विशाल prompt	मॉडल कुछ हिस्सों को ignore करता है	persistent instructions और concrete tasks में बाँटो
verification नहीं	output सुंदर पर fabricated	commands और acceptance criteria परिभाषित करो
context सिर्फ चैट में	हर session में सब दोहरता है	rules repo में लाओ
बहुत व्यापक permissions वाला agent	destructive changes का risk	ownership और minimum permissions
सब कुछ एक subagent में	false parallelism	exploration, implementation और review अलग करो
docs जो चलती नहीं	अच्छी मंशा, पर असर नहीं	docs को workflows और checklists से जोड़ो

AI का आधुनिक उपयोग: checklist

मांग रखने से पहले:

- 1. अंतिम परिणाम स्पष्ट है।
- 2. relevant files या domains का नाम बता सकता हूँ।
- 3. पता है कि क्या नहीं छूना है।
- 4. verify करने का तरीका है।
- 5. पूरे system के बजाय पहला slice स्वीकार कर सकता हूँ।

काम के दौरान:

- 1. risky tasks के लिए छोटे plans मांगता हूँ।
- 2. research, editing और review अलग रखता हूँ।
- 3. file ownership बनाए रखता हूँ।
- 4. बंद करने से पहले diffs पढ़ता हूँ।
- 5. नए decisions दर्ज करता हूँ।

समापन पर:

- 1. मुझे पता है क्या बदला।
- 2. कौन-से tests पास हुए पता है।
- 3. कौन-से tests नहीं चले पता है।
- 4. कौन-से risks बचे हैं पता है।
- 5. reusable knowledge files में दर्ज है।

समापन

आधुनिक AI कौशल का स्थानापन्न नहीं है। जब कौशल व्यवस्थित हो, AI उसे बढ़ाता है। सबसे अधिक मूल्य उसे मिलता है जो secret prompt trick नहीं, बल्कि अस्पष्ट काम को verify होने वाली इकाइयों में बदलना जानता है।

अंतिम नियम सरल है: अगर output महत्वपूर्ण है, तो AI को production system का हिस्सा मानो। उसे context, limits, tools, tests और memory दो। बाकी सब सिर्फ चैट है।

खंड 02

सबएजेंट्स का बुद्धिमान उपयोग

बेहतर डेलीगेशन के लिए मानदंड, पैटर्न और ऑपरेशनल क्लोज़र

यह वॉल्यूम किस काम का है

आधुनिक agents और subagents के साथ काम करना कामों को यूँ ही बाँट देना नहीं है। उपयोगी डेलीगेशन और समय की बर्बादी के बीच अंतर आम तौर पर तीन चीज़ों में होता है: समस्या का आकार, फ्रेमिंग की गुणवत्ता, और परिणाम पर नियंत्रण का स्तर। जब यह सही तरीके से किया जाता है, काम तेज़ होता है, context बेहतर उपयोग होता है, और main thread उन निर्णयों के लिए खाली रहता है जिन्हें सच में मानवीय या आर्किटेक्चरल जजमेंट चाहिए।

यह दस्तावेज़ तकनीकी प्रोजेक्ट्स में subagents उपयोग करने का एक व्यावहारिक तरीका प्रस्तावित करता है। इसका उद्देश्य किसी जादू या पूर्ण ऑटोमेशन को बेचना नहीं है। लक्ष्य अधिक विनम्र और अधिक उपयोगी है: आप काम को समझदारी से बाँट सकें, हर subagent से सिर्फ़ ज़रूरी चीज़ माँग सकें, traceability बनाए रखें, और हर चरण को evidence के साथ बंद करें।

मुख्य विचार यह है: subagent सोच का विकल्प नहीं है। यह सीमित execution unit है जो न्यूनतम context, स्पष्ट ownership और verify की जा सकने वाली output के साथ काम करता है। इनमें से कोई भी हिस्सा गायब हो, तो अभी डेलीगेट न करना बेहतर है।

कब डेलीगेट करना चाहिए

हर काम subagent के लायक नहीं होता। बहुत जल्दी डेलीगेट करने से शोर, दोहराव और ऐसे जवाब मिलते हैं जो सही लगते हैं पर समस्या हल नहीं करते। बहुत देर से डेलीगेट करने पर आप ऐसे दोहराए जाने वाले काम हाथ से करते रहते हैं जो parallel में हो सकते थे।

एक अच्छा व्यावहारिक नियम है: तब डेलीगेट करें जब नीचे की कई शर्तें पूरी हों:

- कार्य स्पष्ट रूप से सीमित हो।
- अपेक्षित परिणाम verify किया जा सके।
- कार्य को सिस्टम के अन्य हिस्सों के साथ cross decisions की ज़रूरत न हो।
- समन्वय की लागत सही ठहराने लायक पर्याप्त repetitive, exploratory या mechanical काम हो।
- आवश्यक context कुछ निर्देशों और ठोस फाइलों में समा जाए।
- subagent के sensitive चीज़ छूने का जोखिम ownership या read-only से नियंत्रित हो।

डेलीगेशन के अच्छे उदाहरण:

- यह ढूँढना कि कोई behavior कहाँ implement है।
- मॉड्यूल्स के बीच data flow मैप करना।
- कोड के किसी क्षेत्र के लिए existing tests पहचानना।
- नोट्स या पहले से बनी संरचना से दस्तावेज़ का draft बनाना।
- सीमित फोल्डर पर targeted validation चलाना।
- ऐसी technical hypotheses जाँचना जिनमें एक साथ बहुत हिस्से edit न करने पड़ें।

डेलीगेशन के खराब उदाहरण:

- “पूरा सिस्टम ठीक कर दो”।
- “आर्किटेक्चर फिर से बना दो”।
- “repo में जाओ और जो दिखे उसे बेहतर कर दो”।
- “acceptance criteria के बिना product decisions ले लो”।
- baseline, metric और scope के बिना “performance optimize कर दो”।

अगर आप यह नहीं बता पा रहे कि कौन-सी file या कौन-सी output चाहिए, तो संभव है कि आप अभी डेलीगेट करने के लिए तैयार नहीं हैं।

Explorer और worker

subagent workflow में सबसे उपयोगी विभाजन **explorer** और **worker** का है।

Explorer

explorer समझने के लिए होता है। इसका काम पढ़ना, मैप करना, तुलना करना, और condensed जानकारी लौटाना है। इसे बदलाव नहीं करने चाहिए, जब तक task स्पष्ट रूप से exploration-with-annotation न हो; और तब भी डिफ़ॉल्ट read-only रखना बेहतर है।

सामान्य उपयोग:

- implementations ढूँढना।
- references ट्रेस करना।
- existing patterns का सार निकालना।
- relevant tests, scripts या configurations पहचानना।
- code छुए बिना विकल्पों की तुलना करना।

इससे क्या माँगना है:

- concrete files quote करें;
- short bullets में summary दें;
- अनावश्यक solutions न propose करें;
- uncertainties पहचान करें;
- क्या confirm नहीं कर पाया, यह बताएं।

जब आपको अभी change surface का वास्तविक आकार नहीं पता, तब explorer बहुत मूल्यवान होता है। edit करने से पहले map समझें।

Worker

worker काम करने के लिए होता है। इसे सीमित objective, स्पष्ट editing zone और verify की जा सकने वाली output criteria मिलती है। यह files edit कर सकता है, commands चला सकता है, या patch तैयार कर सकता है, लेकिन हमेशा explicit ownership के भीतर।

सामान्य उपयोग:

- एक specific function implement करना।
- validation script बनाना।
- documentation file समायोजित करना।
- working branch या files के subset पर hypothesis टेस्ट करना।
- fixtures या test data तैयार करना।

इससे क्या माँगना है:

- सिर्फ authorized files छुएं;

- दूसरों के changes revert न करे;
- क्या बदला, यह समझाए;
- concrete commands से validate करे;
- काम review-ready छोड़े।

सामान्य नियम सरल है: uncertainty घटाने के लिए explorer, पहले से समझे काम को execute करने के लिए worker।

न्यूनतम context

डेलीगेशन में सबसे आम गलतियों में से एक है बहुत ज़्यादा context देना। subagent को "सब कुछ" देना सुरक्षित लगता है, लेकिन अक्सर गुणवत्ता घटती है। context जितना बढ़ेगा, शोर उतना बढ़ेगा, लागत बढ़ेगी, और irrelevant signals मिलाने की संभावना बढ़ेगी।

अच्छा minimal context इन सवालों का जवाब देता है:

1. आप क्या हासिल करना चाहते हैं।
2. सिस्टम का कौन-सा हिस्सा वह छू सकता है।
3. कौन-सी files या paths relevant हैं।
4. उसे क्या नहीं छूना चाहिए।
5. कैसे पता चलेगा कि वह सही तरह से पूरा हुआ।

उपयोगी framing कुछ ऐसी हो सकती है:

- Objective: एक ठोस वाक्य।
- Scope: files और folders।
- Constraints: X न छूना, Y modify न करना, Z के बाहर behavior न बदलना।
- Expected output: summary, patch, findings list या script।
- Verification: tests, commands या manual review।

पूरा project history समझाना ज़रूरी नहीं है। वह हिस्सा समझाना ज़रूरी है जिसकी subagent को जरूरत है ताकि वह बिना improvisation के काम कर सके।

क्या शामिल करें

- सटीक समस्या।
- output format।
- ownership तय करने वाली files।
- validation commands।
- acceptance criteria।

क्या टालें

- ऑपरेशनल relevance के बिना लंबी कहानियाँ।
- विरोधाभासी राय।
- पुरानी clues जो अब लागू नहीं होतीं।
- मानसिक snapshots जिन्हें agent verify नहीं कर सकता।

- “इसे काफी बेहतर कर दो” जैसे खुले निर्देश।

अगर subagent को गलती से बचने के लिए key clarification चाहिए, तो रुककर reframing करना बेहतर है। अगर उसे सिर्फ noisy details चाहिए, तो वे मत दें।

File ownership

ownership वह चीज़ है जो कई agents को एक ही जगह पर चढ़ने से रोकती है। हर task की साफ सीमा होनी चाहिए: worker कौन-सी files edit कर सकता है, कौन-सी सिर्फ पढ़ सकता है, और सिस्टम के कौन-से हिस्से off-limits हैं।

यह सिर्फ hygiene नहीं है। यह parallel काम करते समय integrity बनाए रखने का तरीका है।

अच्छी assignment में शामिल होता है:

- authorized file या folder;
- permission type: read, edit या inspection-only;
- impact limits;
- क्या change invasive माना जाएगा, उसका criterion;
- पुष्टि कि वह दूसरों का काम revert नहीं करेगा।

अच्छे ownership का उदाहरण:

- Worker A: `src/docs/intro.md` और `src/docs/glosario.md`, सिर्फ content edits।
- Worker B: `scripts/validate-docs.mjs`, सिर्फ यह file और इसका associated test।
- Explorer C: `src/` के नीचे कोई भी file, पर बिना edits।

खराब ownership का उदाहरण:

- “जो ज़रूरी लगे edit कर दो”।
- “पूरा module ठीक-ठाक कर दो”।
- “देखो कुछ बेहतर मिले तो कर दो”।

जब ownership स्पष्ट होता है, review भी बेहतर होता है। आपको पता होता है क्या बदला, क्यों बदला, और क्या scope के बाहर है।

Parallelism

subagents तब चमकते हैं जब आप independent काम बाँट सकते हैं। parallelism का मतलब चिंता में एक साथ ज़्यादा काम करना नहीं, बल्कि ऐसे काम अलग करना है जो एक-दूसरे को block न करें।

तीन उपयोगी स्तर हैं:

Exploration parallelism

कई explorers अलग-अलग जानकारी parallel में खोजते हैं।

उदाहरण:

- एक implementation locate करता है;
- दूसरा tests पहचानता है;

- तीसरा similar patterns summarize करता है;
- चौथा risks या dependencies खोजता है।

यह बड़े task की शुरुआत में बहुत मदद करता है। सब कुछ sequentially पढ़ने के बजाय, जल्दी map मिलता है।

Execution parallelism

कई workers अलग क्षेत्रों में बदलाव करते हैं, बशर्ते files overlap न हों।

उदाहरण:

- एक documentation edit करता है;
- दूसरा validation adjust करता है;
- तीसरा examples या fixtures तैयार करता है।

इसमें अनुशासन चाहिए। अगर दो workers files साझा करते हैं, तो parallelism का आधार टूट जाता है और merges या rewrites पर टकराव शुरू हो जाता है।

Verification parallelism

एक subagent implement करता है और दूसरा बाहर से verify करता है।

उदाहरण:

- worker A एक function बदलता है;
- worker B जाँचता है कि relevant tests मौजूद हैं और change conventions नहीं तोड़ता;
- main thread परिणाम integrate करता है।

यह pattern production और evidence अलग करने में उपयोगी है। independent verification confirmation bias कम करती है।

कब parallelize न करें

parallelize करना उचित नहीं जब:

- एक task का निर्णय दूसरे के परिणाम पर निर्भर हो;
- एक ही file कई agents edit करेंगे;
- architecture अभी चर्चा में हो;
- coordination लागत, बचत से अधिक हो;
- एक गलती एक साथ कई हिस्सों को contaminate कर सकती हो।

parallelize करना अपने आप में लक्ष्य नहीं है। यह तभी tool है जब independence सच में मौजूद हो।

Anti-patterns

anti-patterns शुरुआत में productivity जैसे लगते हैं और बाद में operational debt बन जाते हैं।

1. Vague delegation

आप बहुत व्यापक मांग रखते हैं और बहुत generic output पाते हैं। agent खाली जगह assumptions से भर देता है।

सामान्य संकेत: जवाब साफ-सुथरा लगता है पर concrete files या decisions पर नहीं उतरता।

2. Context dumping

आप पूरा repo, पूरी chat, सारे notes दे देते हैं। agent focus खो देता है।

सामान्य संकेत: लंबे जवाब जिनमें relevant हिस्से शोर के साथ मिले हों।

3. Diffuse ownership

किसी को नहीं पता कौन क्या छू सकता है। cross edits, overwrite और confusion शुरू हो जाते हैं।

सामान्य संकेत: “मुझे लगा वह folder free था”।

4. Subagents doing design

subagent strategy गढ़ना शुरू कर देता है जबकि उसे सिर्फ एक concrete slice execute करनी थी।

सामान्य संकेत: मांगा गया change पूरा करने से पहले पूरे सिस्टम को restructure करने का प्रस्ताव।

5. False parallelism

कई tasks “parallel” शुरू होते हैं पर वास्तव में एक ही ज़ोन के लिए प्रतिस्पर्धा करते हैं।

सामान्य संकेत: file conflicts, inconsistent results, या काम दोबारा करना पड़ना।

6. Closure without evidence

subagent कहता है कि काम पूरा है, पर validate कैसे होगा यह नहीं दिखाता।

सामान्य संकेत: न command, न file, न acceptance criteria।

7. Mixing reading with editing without control

agent explore भी करता है और “जब कर ही रहा था” कहकर scope से बाहर चीज़ें भी बदल देता है।

सामान्य संकेत: अनचाहे collateral changes।

golden rule सख्त लेकिन सरल है: अगर किसी task की स्पष्ट review नहीं हो सकती, तो संभवतः delegation गलत हुआ।

Model और effort matrix

हर subagent को एक जैसा model type या reasoning level नहीं चाहिए। व्यावहारिक matrix से सोचना उपयोगी है: task complexity बनाम required effort।

विचार नाम याद करना नहीं, बल्कि काम के अनुसार उचित संयोजन चुनना है।

Mechanical tasks

- उदाहरण: file inspection, search, pattern extraction, simple validations, formatting।
- Suggested model: lightweight या fast।
- Effort: low।
- Objective: गति और कम लागत।

Synthesis tasks

- उदाहरण: findings summarize करना, options compare करना, documentation condense करना, flow map करना।
- Suggested model: lightweight या medium, material density के अनुसार।
- Effort: low से medium।
- Objective: output को अनावश्यक बड़ा किए बिना जानकारी व्यवस्थित करना।

Bounded implementation tasks

- उदाहरण: एक file edit करना, script बनाना, test adjust करना।
- Suggested model: medium।
- Effort: medium।
- Objective: अत्यधिक लागत के बिना अच्छा technical judgment।

Complex debugging tasks

- उदाहरण: multiple-cause bugs, cross integration, intermittent failures।
- Suggested model: stronger model।
- Effort: medium से high।
- Objective: अधिक reasoning क्षमता, कम shortcuts।

Final review tasks

- उदाहरण: महत्वपूर्ण patch review करना, regressions पकड़ना, assumptions पर सवाल करना।
- Suggested model: stronger, या कम से कम implement करने वाले से अलग।
- Effort: medium से high।
- Objective: independence और critical नजरिया।

Practical rule

- अगर task repetitive है, heavy model पर खर्च न करें।
- अगर task fine criteria पर निर्भर है या कई हिस्सों से जुड़ा है, level बढ़ाएँ।
- अगर result महत्वपूर्ण delivery तय करेगा, independent review जोड़ें।

effort "हमेशा high" नहीं होना चाहिए। उसे risk और ambiguity के साथ चलना चाहिए।

Delegation prompts के उदाहरण

बेहतरीन subagent prompts खुले अनुरोध जैसे नहीं लगते। वे अच्छे लिखे tickets जैसे लगते हैं।

Implementation exploration

Objective: find where the autosave flow is implemented.
Role: explorer.
Scope: read-only on `src/` and `tests/`.
Deliverable: list of relevant files, brief flow summary, and risks.
Do not make changes.
If anything is unclear, mark the uncertainty.

Test mapping

Objective: identify existing tests for route validation.
Role: explorer.
Scope: search in `tests/`, `spec/`, and CI scripts.
Deliverable: simple table with file, purpose, and coverage.
Do not edit anything.

Bounded implementation

Objective: add a helper to normalize titles in `src/utils/title.js`.
Role: worker.
Exclusive ownership: only `src/utils/title.js`.
Constraints: do not touch other files, do not change public interfaces.
Deliverable: final code, brief explanation, and verification command.

Document or draft

Objective: write a technical draft on subagent usage.
Role: worker.
Ownership: only `src/02-subagentes-inteligentes.md`.
Style: clear, serious, actionable, in neutral Rioplatense Spanish.
Include criteria, anti-patterns, examples, and a closure checklist.
Do not generate extra files.

Independent verification

Objective: review the change and look for logical errors or scope creep.
Role: explorer.
Scope: read the patch and touched files.
Deliverable: concrete findings, open questions, and validation suggestions.
Do not modify files.

ध्यान दें कि हर मामले में prompt role, scope, deliverable और limits परिभाषित करता है। इससे ambiguity कम होती है और output गुणवत्ता बेहतर होती है।

Closure checklist

डेलीगेशन subagent के जवाब देते ही खत्म नहीं होता। यह तब खत्म होता है जब परिणाम verify हो जाए और main thread बिना संदेह आगे बढ़ सके।

हमेशा उपयोग करने योग्य closure checklist:

- objective एक verify की जा सकने वाली पंक्ति में पूरा हुआ।
- subagent ने authorized ownership के भीतर काम किया।
- scope से बाहर कोई changes नहीं हुए।
- touched files पहचानी गईं।
- output reproducible या reviewable है।
- code changes होने पर validation command मौजूद है।
- document changes होने पर content माँगी गई structure पूरी करता है।
- uncertainties स्पष्ट रूप से बताई गईं।
- parallel work से कोई conflict नहीं है।
- final result को बंद मानने से पहले main thread ने पढ़ लिया।

sensitive tasks के लिए अधिक सख्त संस्करण:

- मैंने diff review किया।
- मैंने verify किया कि external files नहीं छुई गईं।
- मैंने संबंधित validation command चलाई।
- मैंने confirm किया कि hidden assumptions नहीं हैं।
- अगर कुछ बाहर रह गया, तो pending items दर्ज किए।

Recommended process

subagents उपयोग करने का robust flow आम तौर पर ऐसा दिखता है:

1. task और उसके output criteria परिभाषित करें।
2. exploration और execution अलग करें।
3. explicit ownership assign करें।
4. ambiguity और risk के आधार पर model level और effort चुनें।
5. tasks को parallel तभी चलाएँ जब वे overlap न करें।
6. results को main thread में consolidate करें।
7. closure को evidence के साथ verify करें।

अगर task बड़ा है, तो यह flow परतों में दोहराया जा सकता है: पहले explorers, फिर workers, फिर independent review।

अच्छे संकेत

आप सही जा रहे हैं जब:

- subagent कम text में अधिक precision देता है;
- हर output concrete files या commands बताती है;
- main thread बेहतर filtered जानकारी मिलने से तेज़ निर्णय लेता है;
- parallelism समय घटाता है और rework नहीं बढ़ाता;
- integration से पहले errors पकड़े जाते हैं।

आप गलत जा रहे हैं जब:

- आपको लौटी हर चीज़ फिर से interpret करनी पड़ती है;
- surprise changes आते हैं;
- context नियंत्रण से बाहर बढ़ता है;
- validation सिर्फ अंत में होती है और surprises देती है;
- हर subagent को वही base फिर से समझाना पड़ता है।

ऑपरेशनल क्लोज़र

subagents का बुद्धिमान उपयोग मात्रा का नहीं, तरीके का प्रश्न है। delegation की गुणवत्ता agents की संख्या से अधिक scoping, ownership और verification पर निर्भर करती है।

अगर exploration चाहिए, explorer उपयोग करें। अगर execution चाहिए, worker उपयोग करें। अगर एक साथ कई काम करने हैं, काम इस तरह बाँटें कि टकराव न हो। और अगर आप clear ownership परिभाषित नहीं कर सकते, तो अभी डेलीगेट न करें।

सबसे अच्छी प्रैक्टिस “AI को अकेले काम करने देना” नहीं है। बल्कि ऐसी काम-श्रृंखला बनाना है जहाँ हर हिस्से की भूमिका सीमित हो, output जाँची जा सके, और closure स्पष्ट हो। तब subagents एक वादे से आगे बढ़कर सच में उपयोगी tool बनते हैं।

खंड 03

SDD और सपोर्ट संरचना

AI-सक्षम repositories में लागू Specification-Driven Development

SDD, यानी *Specification-Driven Development*, काम करने का ऐसा तरीका है जिसमें specification कोई सजावटी दस्तावेज़ या अलग-थलग नोट नहीं होती: वही development flow का केंद्रीय हिस्सा होती है। बिखरे हुए code से शुरू करने के बजाय, हम पहले साफ़ परिभाषा से शुरू करते हैं कि क्या बनाना है, क्यों बनाना है, उसे कैसे validate करना है, और रास्ते में कौन-से निर्णय दर्ज होंगे।

एक modern repository में, और खासकर जब AI शामिल हो, SDD काम को परतों में व्यवस्थित करता है। पहले समस्या परिभाषित होती है। फिर उसे verifiable specification में बदला जाता है। उसके बाद tasks में तोड़ा जाता है। तभी implementation होता है। और अंत में traceability छोड़ी जाती है: क्या बदला, क्या छोड़ा, क्या test हुआ, और क्या operate करने के लिए तैयार है।

यह खासतौर पर तब उपयोगी है जब टीम AI को copilot, draft generator या analysis assistant की तरह इस्तेमाल करती है। AI बहुत तेज़ी ला सकता है, लेकिन assumptions गढ़ भी सकता है, priorities मिला सकता है, या ऐसा code बना सकता है जो “चलता” तो है पर context का सम्मान नहीं करता। SDD इस पर उपयोगी सीमाएँ लगाता है। AI लक्ष्य का अंदाज़ा नहीं लगाता: वह उसे पढ़ता है। वह acceptance criteria तुरंत नहीं गढ़ता: उन्हें follow करता है। वह निर्णयों को replace नहीं करता: उन्हें document करता है या approval के लिए प्रस्तावित करता है।

SDD क्या हल करता है

SDD एक बहुत ठोस समस्या हल करता है: intention और implementation के बीच की दूरी। बड़े repositories में यह दूरी अक्सर चुपचाप बढ़ती रहती है। issue कुछ कहता है, PR कुछ और हल करता है, code आखिर में कुछ तीसरा करता है, और किसी को नहीं पता चलता कि विचार का सही संस्करण कौन-सा था।

SDD के साथ यह श्रृंखला स्पष्ट हो जाती है:

- **requirements** : कौन-सी जरूरत मौजूद है और किसे उससे फर्क पड़ता है।
- **specs** : expected behavior को कैसे वर्णित किया गया है।
- **decisions** : कौन-से विकल्प evaluate हुए और कौन-सा चुना गया।
- **tasks** : कौन-से ठोस कदम execute करने हैं।
- **acceptance criteria** : कैसे पता चलेगा कि काम सही हुआ।
- **traces** : कौन-सा evidence spec को code और tests से जोड़ता है।
- **runbooks** : failure पर इसे कैसे चलाना या recover करना है।
- **ADRs** : context और consequences के साथ architecture decisions।
- **evaluation notes** : validations, assisted testing या comparative analysis के परिणाम।

फायदा सिर्फ documentation तक सीमित नहीं है। execution भी बेहतर होता है। जब structure अच्छा हो, AI कम बेकार output देता है, टीम बेहतर चर्चा करती है, और changes review करना आसान हो जाता है।

मूल सिद्धांत

SDD का मुख्य नियम सरल है: हर हिस्से का semantic owner होना चाहिए।

- requirement समस्या समझाता है।
- spec behavior परिभाषित करती है।
- decision trade-off समझाता है।
- task काम को व्यवस्थित करती है।
- acceptance scope बंद करती है।
- trace धागा ट्रैक करने देती है।

- runbook operations के लिए तैयार करती है।

अगर सब कुछ एक ही file में मिला दिया जाए, सिस्टम fragile हो जाता है। अगर बिना मानदंड के बहुत ज़्यादा atomize किया जाए, तब भी। SDD संतुलन चाहता है: इतना अलगाव कि हर चीज़ का अपना function रहे, पर इतना नहीं कि एक feature समझने के लिए बीस असंबंधित files खोलनी पड़ें।

सुझाया गया फ़ोल्डर

इस volume के लिए सुझाव है कि SDD material को [.github/orquestador/sdd](https://github.com/orquestador/sdd) के आसपास केंद्रित किया जाए। यह folder specs, decisions और AI-assisted work के governance और execution का core बनता है।

एक संभावित संरचना:

```
.github/orquestador/sdd/
  README.md
  index.md
  requirements/
    000-template.md
    <area>--<id>.md
  specs/
    000-template.md
    <feature>--<id>.md
  decisions/
    000-template-adr.md
    <adr>--<id>.md
  tasks/
    000-template.md
    <issue>--<id>.md
  acceptance/
    000-template.md
    <feature>--<id>.md
  traces/
    000-template.md
    <feature>--<id>.md
  runbooks/
    000-template.md
    <system>--<id>.md
  evaluations/
    000-template.md
    <feature>--<id>.md
  examples/
    sample-issue.md
    sample-spec.md
```

पहले दिन से सभी folders का होना जरूरी नहीं है। महत्वपूर्ण यह है कि architecture scalability के लिए सोचा गया हो और readability बनी रहे।

हर फ़ोल्डर में क्या होगा

README.md

यह entry point है। इसमें बताना चाहिए कि इस repository में SDD क्या है, folder का लक्ष्य क्या है, और इसे कैसे उपयोग करना है। इसमें पूरी theory नहीं, बल्कि system navigate करने के लिए छोटा guide होना चाहिए।

अपेक्षित सामग्री:

- folder का purpose;
- naming convention;
- recommended reading order;
- नई spec या नया decision कैसे बनाना है;
- issues, PRs और general documentation से संबंध।

index.md

यह navigation map है। इसमें active specs, relevant ADRs, latest evaluations और critical runbooks सूचीबद्ध होते हैं। इसमें status भी हो सकता है: draft, in review, approved, implemented, obsolete।

अपेक्षित सामग्री:

- living artifacts का catalog;
- cross-links;
- document-वार status;
- last update date;
- owner या area।

requirements/

यहाँ requirements रहती हैं। ये अभी technical tickets नहीं हैं। ये business, product या operations की needs, pains, objectives या constraints हैं।

अच्छी requirement इन सवालों का उत्तर देती है:

- कौन-सी समस्या मौजूद है;
- किनके लिए;
- अगर हल न हो तो क्या होगा;
- कौन-सी constraints हैं;
- कौन-से signals success दिखाएँगे।

संक्षिप्त उदाहरण:

REQ-014: ग्राहक onboarding में errors कम करना

समस्या: आज forms अथूरा data save करने देते हैं और इससे rework होता है।

उद्देश्य: persist करने से पहले invalid onboarding रोकना।

सीमाएँ: वर्तमान editing flow नहीं टूटना चाहिए।

प्रभाव: कम manual support और reports में कम errors।

specs/

spec behavior का वर्णन करती है। यह सिर्फ समस्या की बात नहीं करती, बल्कि expected system परिभाषित करती है। इसे verifiable होना चाहिए। अच्छी spec inputs, outputs, rules, states, errors, permissions और edge scenarios स्पष्ट करती है।

अपेक्षित सामग्री:

- context;
- scope;
- main behavior;
- edge cases;
- alternate flows;
- dependencies;
- acceptance criteria;
- risks;
- explicit assumptions।

spec को code नहीं लिखना चाहिए, पर pseudo-rules, payload examples, state tables या sequences शामिल कर सकती है।

decisions/

यहाँ ADRs और वे structural decisions रहती हैं जिन्हें याद रखना जरूरी है। अगर spec *क्या* करना है बताती है, तो decision बताता है कि वही रास्ता *क्यों* चुना गया और दूसरा क्यों नहीं।

अपेक्षित सामग्री:

- context;
- considered options;
- chosen decision;
- consequences;
- trade-offs;
- date;
- author या team।

decision का उदाहरण:

ADR-007: validation server पर करें, केवल client पर नहीं

Backend validation को source of truth चुना जाता है।

कारण: client outdated हो सकता है और अकेली barrier के रूप में भरोसेमंद नहीं है।

परिणाम: logic का कुछ भाग duplicate होगा, लेकिन operational risk घटेगा।

tasks/

tasks implementation को छोटे, verifiable steps में तोड़ती हैं। ये spec और code के बीच पुल हैं। अच्छी task कोई vague aspiration नहीं, बल्कि concrete action होती है।

अपेक्षित सामग्री:

- task objective;
- dependencies;
- involved files या modules;
- definition of done;
- expected evidence।

खराब task कहती है: "feature बना दो।" उपयोगी task कहती है: "endpoint Y में field X की validation जोड़ो और integration test से cover करो।"

acceptance/

यह folder acceptance criteria रखता है। इसका काम request और delivery के बीच contract बंद करना है। यह इतनी concrete हो कि reviewer या AI बिना मनमानी व्याख्या के इसे check कर सके।

अपेक्षित सामग्री:

- pass/fail conditions;
- nominal scenarios;
- expected errors;
- observable behavior;
- लागू होने पर non-functional requirements।

traces/

traces requirement, spec, tasks और final result को जोड़ती हैं। इससे पता चलता है कि हर decision कहाँ से आया और कौन-सा evidence उसे support करता है।

अपेक्षित सामग्री:

- link `REQ -> SPEC -> TASK -> PR -> TEST` ;
- coverage summary;
- commits, issues या PRs के references;
- जो scope से बाहर रह गया उस पर notes।

runbooks/

runbooks बताती हैं कि failure होने पर flow को कैसे operate या recover करना है। ये खासकर तब उपयोगी हैं जब spec का परिणाम real-impact feature हो: payments, authentication, jobs, synchronizations, migrations या automated processes।

अपेक्षित सामग्री:

- common symptoms;
- diagnostic steps;
- recovery steps;
- लागू होने पर rollback;
- alert signals;
- contacts या dependencies।

evaluations/

यहाँ evaluation notes जाती हैं: human tests, AI tests, internal benchmarks, before/after comparisons, risk analysis या quality reviews।

अपेक्षित सामग्री:

- क्या evaluate किया गया;
- किस method से;
- क्या result मिला;
- कौन-से defects आए;
- उससे कौन-सा decision लिया गया।

examples/

यह friction घटाने के लिए है। templates और concrete examples adoption को बहुत सुधारते हैं, खासकर जब टीम SDD अभी शुरू कर रही हो।

अपेक्षित सामग्री:

- sample issue;
- sample spec;
- sample ADR;
- sample task;
- sample checklist।

issue या feature का work cycle

स्वस्थ SDD flow PR से शुरू नहीं होता। यह उससे पहले शुरू होता है, जब course correction अभी सस्ता होता है।

1. requirement खोलना

पहले समस्या लिखी जाती है। कविता की जरूरत नहीं, precision की है। requirement को बताना चाहिए कि initiative क्यों है और कौन-सा pain हल करती है।

- -

2. इसे spec में बदलना

फिर expected behavior define होता है। यहाँ उत्तर दिया जाता है:

- क्या input है;
- क्या output है;
- कौन-से rules लागू हैं;
- कौन-सी errors allowed हैं;
- इस version में क्या शामिल नहीं है।

3. decisions दर्ज करना

अगर relevant alternatives हों तो ADR दर्ज करें। इससे दो महीने बाद कोई पहले से तय decision पर फिर वैसी ही बहस नहीं करेगा जैसे कुछ हुआ ही नहीं।

4. tasks में बाँटना

implementation को छोटे tasks में बाँटा जाता है। हर task independently की जा सके, review हो सके और test हो सके।

5. acceptance define करना

code छूने से पहले acceptance criteria लिखी होनी चाहिए। अगर बाद में लिखी जाए, तो अक्सर वे result के हिसाब से ढल जाती हैं और value खो देती हैं।

6. traceability के साथ implement करना

हर change की spec से connection रहनी चाहिए। यह PR references, task notes या trace links से हो सकता है। महत्वपूर्ण यह है कि धागा न टूटे।

7. evaluate करना

अंत में जो हुआ उसे दर्ज करें: कौन-से tests चले, कौन-सी समस्याएँ मिलीं, कहाँ coverage कम है, कौन-सा debt बचा, और final decision क्या रहा।

पूर्ण cycle का एक उदाहरण:

1. **REQ-014** ग्राहक onboarding में errors पहचानता है।
2. **SPEC-014** validations, messages और states परिभाषित करता है।
3. **ADR-007** backend validation चुनता है।
4. **TASK-014.1** validations जोड़ता है।
5. **TASK-014.2** integration tests cover करता है।
6. **AC-014** 5 acceptance criteria परिभाषित करता है।
7. **TRACE-014** issue, PR और tests को link करता है।
8. **EVAL-014** परिणाम और open points summarize करता है।

छोटी templates

Requirement

REQ-XXX: छोटा शीर्षक

समस्या:

उद्देश्य:

प्रभावित उपयोगकर्ता:

सीमाएँ:

न करने का जोखिम:

Spec

SPEC-XXX: छोटा शीर्षक

संदर्भ:

दायरा:

दायरे से बाहर:

अपेक्षित व्यवहार:

edge cases:

acceptance criteria:

ADR

ADR-XXX: छोटा निर्णय

संदर्भ:

विकल्प:

निर्णय:

परिणाम:

Task

TASK-XXX: ठोस कार्रवाई

क्या करना है:

संभावित files:

dependencies:

कैसे validate होगा:

Acceptance

```
# AC-XXX: feature या spec

- Given ...
- When ...
- Then ...
```

Trace

```
# TRACE-XXX: छोटा शीर्षक

REQ:
SPEC:
TASKS:
PR:
TESTS:
NOTAS:
```

Runbook

```
# RUN-XXX: system या flow

लक्षण:
निदान:
रिकवरी:
Rollback:
```

Evaluation notes

```
# EVAL-XXX: छोटा शीर्षक

क्या test किया गया:
मापदंड:
परिणाम:
समस्याएँ:
निर्णय:
```

AI के साथ SDD कैसे उपयोग करें

AI documentation की value बदल देता है। पहले spec मुख्यतः इंसानों के लिए थी। अब यह assistants के लिए context contract भी है।

AI के साथ एक अच्छा flow यह है:

- AI requirement पढ़कर समस्या summarize करे;
- AI शुरूआती spec प्रस्तावित करे;
- इंसान scope और priorities validate करे;
- AI tasks को तोड़े और tests सुझाए;
- इंसान decisions approve या correct करे;
- AI implementation और traceability review में मदद करे;
- इंसान final acceptance verify करे।

यह तब बेहतर चलता है जब repository में artifacts stable और readable हों। spec बिखरी होगी तो AI के जवाब भी बिखरेंगे। acceptance vague होगी तो AI gaps assumptions से भर देगा। ADRs नहीं होंगी तो decisions खो जाएँगे और context हर बार reset होगा।

कुंजी यह है कि AI को reasoning तेज़ करने के लिए उपयोग करें, उसे replace करने के लिए नहीं। SDD AI को explicit आधार पर काम करने देता है। “मेरे लिए feature बना दो” की जगह “इस spec और इन acceptance criteria के आधार पर implementation तोड़ो और risks चिन्हित करो” कहना बेहतर है। भाषा का यह बदलाव output quality को बहुत सुधारता है।

Anti-patterns

सब कुछ एक ही file में मिलाना

जब requirements, spec, task और decision बिना structure के साथ रहते हैं, document उपयोग के लायक नहीं रहता। किसी को नहीं पता कौन-सा भाग source of truth है।

अस्पष्ट specifications

“यह intuitive होना चाहिए” या “यह अच्छे से चलना चाहिए” जैसी पंक्तियाँ तब तक बेकार हैं जब तक उन्हें verifiable criteria में न बदला जाए। अगर test नहीं हो सकता, तो यह अधूरा है।

बहुत बड़े tasks

“पूरा flow implement करो” जैसी task अक्सर ऐसी PR में बदलती है जिसे review करना कठिन होता है और traceability debt छोड़ती है।

देर से लिखे ADRs

context भूल जाने के बाद decisions लिखना उनकी value खत्म कर देता है। ADR तभी काम का है जब वह वास्तविक चर्चा को संरक्षित करे।

acceptance को result के हिसाब से फिर लिखना

अगर acceptance criteria code result से मेल कराने के लिए बदली जाती हैं, तो वे contract नहीं रहतीं, justification बन जाती हैं।

खाली traces

सिर्फ “PR में है” कहना काफी नहीं। अगर मूल requirement तक धागा नहीं जाता, तो traceability मौजूद नहीं है।

ghost runbooks

ऐसी runbook जिसे emergency में कोई चला न सके, बेकार है। वह छोटी, स्पष्ट और actionable होनी चाहिए।

निष्कर्ष बिना evaluations

कुछ मापकर यह न बताना कि उससे क्या decision लिया गया, अधूरा काम है। evaluation को ठोस निष्कर्ष छोड़ना चाहिए।

Practical checklist

SDD के तहत feature बंद करने से पहले यह जाँचना उपयोगी है:

- requirement लिखी हुई है और उसमें समस्या, उद्देश्य और impact है;
- spec verifiable behavior define करती है;
- scope में क्या शामिल नहीं है, साफ़ है;
- relevant decisions documented हैं;
- tasks छोटे steps में बाँटी गई हैं;
- acceptance criteria concrete हैं;
- issue, spec, PR और tests के बीच traceability है;
- अगर flow operations को प्रभावित करता है, runbook मौजूद है;
- evaluation या validation notes दर्ज हैं;
- document, code और tests में विरोधाभास नहीं है;
- repository oral memory पर निर्भर हुए बिना context reconstruct कर सकती है।

रखरखाव की सिफारिश

इस structure को bureaucratic बनने से बचाने के लिए तीन नियम बनाए रखना उपयोगी है:

1. कम लिखो, पर उपयोगी लिखो;
2. code की गति के साथ ही update करो;
3. हर feature evidence के साथ बंद करो।

मकसद सिर्फ दस्तावेज़ बनाना नहीं है। मकसद है कि documentation काम की infrastructure बने। AI-enabled repo में यह infrastructure उत्पाद का हिस्सा है। यह errors घटाती है, review बेहतर करती है, और knowledge को context rotation के बावजूद जीवित रखती है।

समापन

SDD कोई naming trend नहीं है, न engineering का जादुई विकल्प। यह एक अनुशासन है ताकि code intention को धुंधला करने से पहले intention व्यवस्थित हो जाए। जिन repositories में AI writing, review या analysis में भाग लेता है, वहाँ यह अनुशासन और ज़रूरी हो जाता है क्योंकि ambiguity की लागत कई गुना बढ़ती है।

[.github/orquestador/sdd](https://github.com/orquestador/sdd) की संरचना एक सरल प्रस्ताव देती है: सही हिस्सों को अलग करो ताकि हर हिस्सा अपना काम करे और सब मिलकर readable system बनें। Requirements, specs, decisions, tasks, acceptance criteria, traces, runbooks,

ADRs और evaluation notes सजावटी folders नहीं हैं। ये वह mechanism हैं जिनसे विचार समस्या से implementation तक जाता है और रास्ते में context नहीं खोता।

अगर repository यह काम करने का तरीका अपनाती है, तो हर feature भरोसे की छलांग नहीं रहती। वह एक traceable, reviewable और improvable process बन जाती है। और AI-assisted stack में इसकी कीमत किसी भी तात्कालिक shortcut से ज़्यादा है।

खंड 04

AGENTS.md, .github और Prompt-स्टाइल Commands

agents, SDD, Copilot, workflows और reusable prompts के लिए तैयार repository कैसे बनाएं

यह खंड क्यों मौजूद है

एक modern repository को इस बात पर निर्भर नहीं होना चाहिए कि इंसान हर बार AI assistant खोलते समय सारे नियम याद रखे। प्रोजेक्ट के नियम फाइलों में रहने चाहिए। कुछ नियम इंसानों के लिए होते हैं, कुछ agents के लिए, कुछ Copilot के लिए, कुछ CI के लिए, और कुछ SDD प्रोसेस के लिए। जब ये परतें मिल जाती हैं, तो AI अधूरे context के साथ काम करता है और टीम बार-बार वही गलतियां सुधारती रहती है।

यह खंड उन repositories के लिए एक practical structure प्रस्तावित करता है जो AI को गंभीरता से उपयोग करना चाहते हैं:

- 1. `AGENTS.md` को code agents के operational contract के रूप में।
- 2. `.github/copilot-instructions.md` को Copilot के general instructions के रूप में।
- 3. `.github/instructions/*.instructions.md` को path-specific instructions के रूप में।
- 4. `.github/prompts/*.prompt.md` को reusable prompt-स्टाइल commands के रूप में।
- 5. `.github/orquestador/sdd/*` को specifications, decisions और traceability के सिस्टम के रूप में।
- 6. `.github/workflows/*` को एकमात्र executable automation layer के रूप में।
- 7. `.github/orquestador/pipelines/catalog.md` को governance catalog के रूप में, न कि दूसरी automation engine के रूप में।

उद्देश्य repository को ceremony से भरना नहीं है। उद्देश्य यह है कि हर फाइल का उद्देश्य स्पष्ट हो और कोई agent तीन सवाल जल्दी उत्तर दे सके: यह कौन-सा प्रोजेक्ट है, यहां काम कैसे होता है, और बदलाव सही है यह कैसे verify होता है।

जिम्मेदारियों का मानचित्र

फाइल या फोल्डर	मुख्य audience	जिम्मेदारी
<code>README.md</code>	नए लोग	प्रोडक्ट, इंस्टॉलेशन और उपयोग समझाना
<code>AGENTS.md</code>	code agents	operational rules, commands, permissions, style और closure
<code>.github/copilot-instructions.md</code>	GitHub Copilot	repository की persistent general instructions
<code>.github/instructions/*.instructions.md</code>	zone-wise Copilot	specific paths पर लागू नियम
<code>.github/prompts/*.prompt.md</code>	टीम और assistants	बार-बार होने वाले tasks के invocable prompts
<code>.github/orquestador/context/*</code>	टीम और agents	stable context: product, architecture, glossary
<code>.github/orquestador/sdd/*</code>	टीम और agents	specs, decisions, tasks, traceability, runbooks
<code>.github/orquestador/pipelines/catalog.md</code>	Maintainers	workflows, permissions, risks और owners की inventory
<code>.github/workflows/*</code>	GitHub Actions	वास्तविक automation: CI, reviewers, validations

Golden rule: context file को execution का दिखावा नहीं करना चाहिए। Workflow execute करता है। Catalog document करता है। Prompt दिशा देता है। `AGENTS.md` agent behavior govern करता है। यह separation बनाए रखने से confusing systems से बचाव होता है।

अनुशंसित संरचना

AI, SDD और GitHub-first तरीके से काम करने वाली repo के लिए एक उचित आधार:

```
repo/
  AGENTS.md
  README.md
  src/
  tests/
  docs/
  .github/
    copilot-instructions.md
  instructions/
    frontend.instructions.md
    backend.instructions.md
    tests.instructions.md
  prompts/
    plan-feature.prompt.md
    write-spec.prompt.md
    review-pr.prompt.md
    generate-tests.prompt.md
    write-adr.prompt.md
    qa-harness.prompt.md
  workflows/
    ci.yml
    pr-reviewer.yml
  orquestador/
    README.md
    context/
      product.md
      architecture.md
      glossary.md
      constraints.md
    sdd/
      README.md
      requirements/
      specs/
      decisions/
      tasks/
      traces/
      evals/
      runbooks/
    pipelines/
      catalog.md
    policies/
      permissions.md
      safety.md
```

हर प्रोजेक्ट को पहले दिन से सब कुछ नहीं चाहिए। लेकिन mental architecture होना उपयोगी है। एक MVP `AGENTS.md`, `context/product.md`, `sdd/specs/`, `prompts/` और `workflows/ci.yml` से शुरू हो सकता है। बाकी तब जोड़ा जाए जब वास्तविक repetition दिखे।

अच्छा **AGENTS.md** कैसे लिखें

AGENTS.md वह फाइल है जो agent को बताती है कि repository में operate कैसे करना है। यह landing page नहीं है। यह product vision नहीं है। यह इंसानों के लिए लंबा manual नहीं है। यह operational guide है।

इसे इन सवालों का जवाब देना चाहिए:

1. Stack क्या है।
2. हर महत्वपूर्ण चीज कहाँ है।
3. install, test, validate और run के लिए कौन-से commands हैं।
4. कौन-सी files या folders sensitive हैं।
5. बदलावों की expected style क्या है।
6. कौन-सी permissions या actions निषिद्ध हैं।
7. task बंद कैसे करना है।

Base example:

```
# AGENTS.md

## Project Snapshot

Este repo contiene una app Node.js + TypeScript con API en `src/server`,
frontend en `src/web` y tests en `tests`.

## Commands

- Instalar: `npm ci`
- Desarrollo: `npm run dev`
- Tests: `npm test`
- Typecheck: `npm run typecheck`
- Build: `npm run build`

## Working Rules

- Mantener cambios acotados al pedido.
- No modificar migraciones antiguas sin pedir permiso.
- No tocar secretos ni archivos `.env`.
- Preferir tests cerca del comportamiento cambiado.
- Si un comando falla, reportar el error exacto y el proximo paso.

## SDD

- Specs: `.github/orquestador/sdd/specs/`
- Decisiones: `.github/orquestador/sdd/decisions/`
- Tareas: `.github/orquestador/sdd/tasks/`
- Trazabilidad: `.github/orquestador/sdd/traces/`

## Completion

Antes de cerrar, informar:

- Archivos modificados.
- Pruebas ejecutadas.
- Pruebas no ejecutadas y por que.
- Riesgos residuales.
```

सामान्य गलती यह है कि `AGENTS.md` बहुत दार्शनिक लिख दिया जाता है। Agent को "best practices अपनाओ" जैसी पंक्तियां नहीं चाहिए। उसे commands, paths, limits और criteria चाहिए।

निर्देशों की hierarchy

निर्देशों की precedence होती है। यूज़र का explicit request, repo के general rule से ज्यादा वज़न रखता है। किसी subfolder के पास वाला `AGENTS.md`, root `AGENTS.md` के नियमों को specialize कर सकता है। System या platform instructions सबसे ऊपर priority रखते हैं।

इसे practical तरीके से ऐसे समझें:

```
System / platform
> बातचीत में User
  > touched file के सबसे करीब AGENTS.md
    > Root AGENTS.md
      > सहायक documentation
```

यह monorepos में खास तौर पर उपयोगी है। Root `AGENTS.md` global काम का तरीका बता सकता है, जबकि `packages/mobile/AGENTS.md` mobile-specific commands और conventions तय करता है।

`.github/copilot-instructions.md`

GitHub, Copilot के लिए repository custom instructions को `.github/copilot-instructions.md` फाइल में document करता है। इसका काम है Copilot को repository के अंदर काम करते समय persistent context देना।

यह फाइल `AGENTS.md` से छोटी होनी चाहिए। हर command दोहराने की जरूरत नहीं। यह style, architecture, response preferences और expected validations पर फोकस कर सकती है।

Example:

```
# Copilot Instructions

Este proyecto usa TypeScript estricto. Evitar `any` salvo justificacion.
Preferir funciones puras en la capa de dominio.
No crear dependencias nuevas sin explicar el motivo.
Cuando sugieras codigo, incluir pruebas relevantes.
Si falta contexto de negocio, preguntar o marcar supuesto.
```

इसे general quality guide करने के लिए उपयोग करें। इसे बहुत बड़ा document न बनाएं। अगर Copilot को बहुत ज्यादा general rules मिलें, तो conflicts या कुछ content ignore होने की संभावना बढ़ती है।

`.github/instructions/*.instructions.md`

Path-based instructions आपको यह कहने देते हैं: "जब repo के इस हिस्से पर काम करो, ये नियम लागू करो।" GitHub `.github/instructions` के अंदर `NAME.instructions.md` pattern को `applyTo` frontmatter के साथ document करता है।

Backend example:

```

---
applyTo: "src/server/**/*.ts,tests/server/**/*.ts"
---

# Backend Instructions

- Validar entradas en la frontera HTTP.
- Mantener reglas de negocio fuera de handlers.
- No acceder a la base de datos desde controllers.
- Agregar tests de casos borde para errores 4xx y 5xx.

```

Frontend example:

```

---
applyTo: "src/web/**/*.tsx,src/web/**/*.css"
---

# Frontend Instructions

- Mantener componentes accesibles por teclado.
- Evitar texto que se desborde en mobile.
- Usar componentes existentes antes de crear nuevos.
- No introducir cambios visuales globales sin justificar.

```

फायदा है precision। frontend की rule backend में फैलने के बजाय, हर zone को वही मिलता है जो उसे चाहिए।

[.github/prompts/*.prompt.md](#)

Prompt files reusable commands होते हैं। GitHub इन्हें `.prompt.md` extension वाली Markdown files के रूप में document करता है, आम तौर पर `.github/prompts` के अंदर। ये उन tasks के लिए हैं जिन्हें टीम बार-बार करती है: feature plan करना, PR review करना, tests generate करना, ADR लिखना, API document करना, या onboarding तैयार करना।

इन्हें versioned commands की तरह treat करें। अगर कोई prompt बेहतर हो, तो PR में review करें। fail हो तो adjust करें। काम न आए तो remove करें।

`plan-feature.prompt.md` example:

```
---  
description: "Convertir una idea de producto en plan SDD por slices"  
---
```

Usa el contexto de:

- [producto](../orquestador/context/product.md)
- [arquitectura](../orquestador/context/architecture.md)

Entrada del usuario:

`${input:feature:Describe la feature}`

Producir:

1. Problema y objetivo.
2. No objetivos.
3. Supuestos.
4. Requisitos funcionales.
5. Criterios de aceptacion.
6. Slices de implementacion.
7. Tests recomendados.
8. Riesgos y preguntas abiertas.

No inventes reglas de negocio. Marca lo desconocido.

`review-pr.prompt.md` example:

```
---  
description: "Revisar un PR con foco en bugs, riesgos y pruebas"  
---
```

Revisa los cambios del PR actual.

Prioridad:

1. Bugs o regresiones.
2. Riesgos de seguridad o permisos.
3. Falta de tests para comportamiento nuevo.
4. Inconsistencias con SDD o ADRs.

Salida:

- Findings con archivo y línea cuando sea posible.
- Preguntas abiertas.
- Resumen breve.

No hagas comentarios de estilo si no afectan mantenimiento o comportamiento.

`qa-harness.prompt.md` example:

```
---
description: "Disenar un harness de evaluacion para una capacidad de IA"
---
```

```
Capacidad a evaluar:
${input:capability:Describe la capacidad}
```

```
Disena:
```

1. Fixtures de entrada.
2. Salidas esperadas o criterios.
3. Rubrica de scoring.
4. Casos negativos.
5. Script CLI minimo.
6. Como integrarlo en CI.
7. Riesgos de falsos positivos.

ये फाइलें टीम के ज्ञान को invocable बनाती हैं। ये human judgment को replace नहीं करती, लेकिन variability कम करती हैं।

.github/orquestador

`.github/orquestador` फोल्डर काम करने के सिस्टम का home बन सकता है। नाम जादुई नहीं है। महत्वपूर्ण यह है कि इसकी जिम्मेदारी स्पष्ट हो: context, SDD, policies और catalogs को एक जगह लाना ताकि humans और agents को दिशा मिले।

एक उपयोगी convention:

<code>.github/orquestador/</code>	
<code>README.md</code>	Repo operating system का index
<code>context/</code>	Stable context
<code>sdd/</code>	Specifications और traceability
<code>prompts/</code>	Internal prompts, अगर <code>`.github/prompts`</code> use न हो
<code>pipelines/</code>	Workflows और automations का catalog
<code>policies/</code>	Permissions, security, risk criteria

अगर टीम GitHub Copilot बहुत उपयोग करती है, तो tool compatibility के लिए `.github/prompts` बनाए रखना बेहतर है और `.github/orquestador` को context और SDD के लिए रखें। अगर कोई दूसरा agent उपयोग होता है, तो `orquestador/prompts` भी हो सकता है। महत्वपूर्ण है बिना जरूरत duplication न करना।

Repo के अंदर SDD

SDD का अर्थ है specifications से काम करना, impulses से नहीं। AI वाली repository में SDD का critical role है: यह agent को request की creative interpretation implement करने से रोकता है।

एक minimal spec में यह शामिल होना चाहिए:

1. Problem.

2. Objective.
3. Non-objectives.
4. Requirements.
5. Acceptance criteria.
6. Edge cases.
7. Technical impact.
8. Slice-wise plan.
9. Tests.
10. Issue, PR और decisions के साथ traceability.

Path example:

```
.github/orquestador/sdd/specs/2026-05-08-reassignar-reclamo.md
```

Header example:

```
# Reassignar reclamo a otro equipo

Estado: Draft
Owner: Producto / Backend
Issue: #123
PRs: pendiente

## Problema

Los operadores no pueden mover un reclamo cuando fue derivado al equipo incorrecto.

## Criterios de aceptacion

- Un operador autorizado puede reassignar el reclamo.
- La reasignacion queda auditada.
- El equipo anterior y el nuevo quedan visibles en el historial.
- Si el reclamo esta cerrado, no puede reasignarse.
```

जब agent implement करे, उसे इस spec से स्पष्ट समझ आना चाहिए कि "done" का मतलब क्या है।

Workflows को एकमात्र executable layer रखना

अगर repository में GitHub मुख्य surface है, तो [.github/workflows](https://github.com/workflows) को एकमात्र executable automation layer रखना बेहतर है। बाकी चीजें document, guide या govern कर सकती हैं। यह separation दो समस्याएं रोकता है: duplicate automations और यह भ्रम कि execution वास्तव में कहाँ हो रहा है।

शुरुआत के लिए conservative pattern:

```

name: Safe PR Reviewer

on:
  pull_request:
    types: [opened, synchronize, reopened]

permissions:
  contents: read
  pull-requests: read
  issues: write

concurrency:
  group: pr-reviewer-${{ github.event.pull_request.number }}
  cancel-in-progress: true

jobs:
  review:
    runs-on: ubuntu-latest
    steps:
      - name: Comment with checklist
        uses: actions/github-script@v7
        with:
          script: |
            const body = [
              "Revision automatica inicial:",
              "- Verificar tests.",
              "- Confirmar criterios SDD.",
              "- Revisar permisos y secretos.",
              "- Mantener cambios acotados."
            ].join("\n");
            await github.rest.issues.createComment({
              owner: context.repo.owner,
              repo: context.repo.repo,
              issue_number: context.payload.pull_request.number,
              body
            });

```

यह workflow checkout नहीं करता, PR code execute नहीं करता, और conservative तरीके से comment करता है। यह पहला अच्छा automation है क्योंकि यह बिना बड़ा risk surface खोले व्यवस्था देता है।

Pipeline catalog

Catalog execute नहीं करता। यह document करता है। इसे यह जवाब देना चाहिए:

1. कौन-सा workflow मौजूद है।
2. उसे कौन-सा event trigger करता है।

3. वह कौन-सी permissions उपयोग करता है।
4. उसके क्या risks हैं।
5. उसका maintainer कौन है।
6. वह कौन-सा output देता है।
7. उसे क्या करने की अनुमति नहीं है।

Example:

```
# Pipeline Catalog

## safe-pr-reviewer

- Archivo: `.github/workflows/pr-reviewer.yml`
- Evento: `pull_request`
- Permisos: `contents: read`, `pull-requests: read`, `issues: write`
- Ejecuta codigo del PR: no
- Output: comentario con checklist
- Owner: Platform
- Riesgo principal: ruido en PRs si las reglas no se mantienen
- Estado: activo
```

जब कोई पूछे "इस repo में कौन-सी automations हैं?", तो catalog जवाब देता है। जब कोई पूछे "असल में क्या execute होता है?", जवाब फिर भी `.github/workflows` ही है।

हिस्से आपस में कैसे जुड़ते हैं

एक complete flow यह हो सकता है:

1. इंसान एक issue बनाता है।
2. वह `plan-feature.prompt.md` चलाकर idea को spec में बदलता है।
3. spec को `.github/orquestador/sdd/specs/` में सेव करता है।
4. agent `AGENTS.md`, context और spec पढ़ता है।
5. agent एक scoped slice implement करता है।
6. वह `AGENTS.md` में बताए tests चलाता है।
7. वह PR खोलता है।
8. `pr-reviewer.yml` checklist comment करता है।
9. दूसरा agent या इंसान `review-pr.prompt.md` उपयोग करता है।
10. SDD में traceability अपडेट की जाती है।

महत्वपूर्ण बात यह है कि हर कदम traces छोड़ता है। टीम को chat में क्या बात हुई, यह याद रखने पर निर्भर नहीं रहना पड़ता।

सामान्य गलतियां

गलती	परिणाम	सुधार
बहुत बड़ा <code>AGENTS.md</code>	agent कुछ हिस्से ignore करता है	इसे operational रखें और लंबे docs को link करें
पांच फाइलों में नियम दोहराना	conflicting instructions	layer-wise owner तय करें
unversioned prompts	हर व्यक्ति अलग variant उपयोग करता है	इन्हें <code>.github/prompts</code> में रखें
broad permissions वाले workflows	अनावश्यक risk	प्रति workflow minimum permissions
execution का वादा करता catalog	operational confusion	catalog document करता है, workflows execute करते हैं
acceptance criteria बिना specs	ambiguous implementations	हर spec को tests और examples से close करें

Bootstrap checklist

repo को तैयार छोड़ने के लिए:

- वास्तविक commands के साथ `AGENTS.md` बनाएं।
- छोटा `.github/copilot-instructions.md` बनाएं।
- `.github/instructions/` तभी बनाएं जब path-wise rules हों।
- 3 से 5 उपयोगी prompts के साथ `.github/prompts/` बनाएं।
- `.github/orquestador/context/product.md` बनाएं।
- `.github/orquestador/context/architecture.md` बनाएं।
- `.github/orquestador/sdd/README.md` बनाएं।
- `specs/`, `decisions/`, `tasks/`, `traces/`, `evals/`, `runbooks/` बनाएं।
- `.github/orquestador/pipelines/catalog.md` बनाएं।
- verify करें कि `.github/workflows` ही automation execute करने वाली एकमात्र layer है।
- एक test PR बनाकर पुष्टि करें कि instructions समझने योग्य हैं।

सत्यापित स्रोत

GitHub, Copilot के लिए repository instructions document करता है, जिनमें `.github/copilot-instructions.md`, `.github/instructions/*.instructions.md`, और agents instructions के लिए `AGENTS.md` का उपयोग शामिल है:

- GitHub Docs - Adding repository custom instructions for GitHub Copilot
- GitHub Docs - Prompt files
- GitHub Docs - Your first prompt file

- [openai/agents.md](#)

नोट: 2026-05-08 को इन स्रोतों की जांच के समय Copilot prompt files को public preview के रूप में document किया गया था, इसलिए इन्हें किसी enterprise में rigid standard बनाने से पहले official documentation दोबारा देखना उचित है।

समापन

लक्ष्य एक प्रभावशाली `.github` फ़ोल्डर बनाना नहीं है। लक्ष्य यह है कि repo में आने वाला हर agent कम guesswork, ज्यादा verification और बेहतर memory के साथ काम कर सके। `AGENTS.md` operations के नियम देता है। SDD product और technical contract देता है। Prompt files दोहराए जाने वाले tasks को commands में बदलते हैं। Workflows validations चलाते हैं। Catalog सिस्टम समझाता है।

जब ये हिस्से align होते हैं, तो AI एक loose chat नहीं रहता, बल्कि work infrastructure बन जाता है।

खंड 05

Prompt Engineering और Harness Engineering

अलग-अलग prompts से versioned, evaluable और production-ready systems तक

एक उपयोगी experiment और एक भरोसेमंद system के बीच का फर्क सिर्फ अच्छा prompt लिखने में नहीं है। एक isolated prompt किसी एक काम को हल कर सकता है, लेकिन एक गंभीर product को और चीजें चाहिए: versions, test cases, evaluation criteria, observability, और security rules। यही पूरा सेट किसी idea को operable capability में बदलता है।

यह वॉल्यूम एक सरल विचार से शुरू होता है: prompt को app, notebook या comment में चिपकी हुई loose string की तरह नहीं रहना चाहिए। अगर कोई instruction business के लिए महत्वपूर्ण है, तो उसे auditable, comparable, testable, और disciplined तरीके से deployable होना चाहिए। यहीं harness engineering आती है: उस environment का design जो prompt को execute, measure, और control करता है।

प्रेक्टिस का फोकस बदल जाता है। सवाल सिर्फ "model को क्या कहूं?" नहीं रहता, बल्कि "इस task को हर बार same criteria के साथ कैसे चलाऊं, verify कैसे करूं, और system बढ़ने पर इसे टूटने से कैसे बचाऊं?" बन जाता है। यही artisanal prompt engineering से productive prompt engineering की transition है।

जब prompt production में जाता है तो क्या बदलता है

demo में prompt वही text हो सकता है जिसने आज अच्छा result दिया। production में वही text पर्याप्त नहीं रहता क्योंकि नई conditions आ जाती हैं:

1. model बदलता है।
2. temperature बदलता है।
3. user context बदलता है।
4. input data बदलता है।
5. system maintain करने वाली team बदलती है।
6. legal, security, या brand requirements जुड़ती हैं।

ऐसा होने पर समस्या सिर्फ "response quality" नहीं होती। असली समस्या control की होती है। आपको पता होना चाहिए कि कौन-सी instruction इस्तेमाल हुई, किस version के साथ, किन inputs पर, किन tools के साथ, और behavior अभी भी expected सीमा में है या नहीं।

इसीलिए production prompt system में आमतौर पर ये हिस्से होते हैं:

- persistent instructions, जो identity और stable behavior define करें;
- task prompts, जो specific request solve करें;
- fixtures, जो well-defined input cases represent करें;
- golden tests, जो expected output या observable properties fix करें;
- evals, जो rubric के साथ quality मापें;
- regression tests, जो degradation पकड़ें;
- permissions और security rules;
- context में result review के लिए logging और observability।

ये decorative layers नहीं हैं। assistant, classifier, writer, router, या agent पर भरोसा करने के लिए यही minimum infrastructure है।

एक अच्छे prompt की anatomy

अच्छा prompt सिर्फ लंबा या छोटा होने से तय नहीं होता। वह तब उपयोगी होता है जब ambiguity घटाए, priorities क्रम में रखे, और data missing होने पर क्या करना है यह स्पष्ट करे। प्रैक्टिस में best prompts की structure मिलती-जुलती होती है।

1. स्पष्ट objective

model को पता होना चाहिए कि वह कौन-सी problem solve कर रहा है। सिर्फ "user की मदद करो" काफी नहीं है। objective को operational terms में लिखना बेहतर है।

उदाहरण:

तुम्हारा काम informal requests को clear, actionable और complete work tickets में बदलना है।

2. operation context

prompt को बताना चाहिए कि वह किस environment में इस्तेमाल हो रहा है और उसकी limits क्या हैं। support, sales agent, और internal classifier के लिए writing समान नहीं होती।

उदाहरण:

यह assistant operations team में इस्तेमाल होता है। clarity, traceability, और professional language को priority दो।

3. quality criteria

अगर आप quality define नहीं करते, model खुद बना लेता है। अच्छा prompt यह स्पष्ट करता है कि क्या value है: precision, concision, coverage, tone, safety, format।

उदाहरण:

response concrete होनी चाहिए, data invent नहीं करना चाहिए, और observable facts को assumptions से अलग करना चाहिए।

4. constraints

constraints comfortable लेकिन useless responses से बचाते हैं। इसमें format, length, available tools, languages, exclusions, और security rules आते हैं।

उदाहरण:

अगर जानकारी uncertain हो तो tables का उपयोग न करो। permissions assume न करो। confirmation के बिना actions execute न करो।

5. uncertainty policy

mature prompt बताता है कि context missing होने पर क्या करना है। इससे hallucinations और shaky responses कम होते हैं।

उदाहरण:

अगर `critical data missing` है तो सिर्फ एक `clarification question` पूछो। अगर `data non-critical` है तो `explicit assumption` के साथ आगे बढ़ो।

6. output format

output इंसान या system की अगली layer दोनों के लिए आसान होनी चाहिए। JSON चाहिए तो schema बताओ। bullets चाहिए तो साफ कहो। template चाहिए तो define करो।

उदाहरण:

हमेशा लौटाओ:

1. Summary
2. Risks
3. Next steps

7. examples

examples सजावट नहीं हैं; वे semantic anchors हैं। वे tone, detail level, और boundary decisions को stabilize करते हैं। खासकर तब उपयोगी होते हैं जब task में format या criteria ambiguity हो।

एक अच्छा prompt आमतौर पर instructions के साथ 1 या 2 compact examples रखता है। इसे overload नहीं करना चाहिए: बहुत ज्यादा examples noise बढ़ाते हैं।

persistent instructions बनाम task prompts

एक common गलती है सब कुछ एक ही instruction में मिला देना। इससे system fragile हो जाता है। उपयोगी split यह है:

- persistent instructions: जो tasks के बीच नहीं बदलना चाहिए;
- task prompts: जो हर request में बदलते हैं;
- dynamic context: user data, files, session state, tools, temporary memory।

persistent instructions

यह identity और policy layer है। यह role, tone, criteria priority, safety limits, language, और expected behavior define करती है।

उदाहरण:

तुम एक operations assistant हो। तुम neutral Rioplatense Spanish में clear और serious style में जवाब देते हो। तुम precision, safety और actionable steps को prioritize करते हो।

task prompts

ये किसी specific execution के लिए point instructions हैं। इन्हें short, specific, और result-focused होना चाहिए।

उदाहरण:

इस text के आधार पर अधिकतम 180 शब्दों का executive summary लिखो और अंत में 3 concrete risks दो।

practical rule

अगर कोई instruction हर case में repeat होती है, तो वह persistent layer में जाती है। अगर वह हर execution में बदलती है, तो task layer में जाती है। अगर वह user input में है, तो उसे policy के साथ mix मत करो। इस separation से errors कम होते हैं और system maintainable बनता है।

Fixtures: behavior test करने के लिए concrete cases

fixture एक controlled input case है जो realistic situation को represent करता है। prompt engineering में fixtures जरूरी हैं क्योंकि ये दिखाते हैं कि system typical, rare, और risky scenarios में कैसा behave करता है।

सिर्फ एक happy example पर्याप्त नहीं है। serious harness को ऐसे fixtures चाहिए जो cover करें:

- clean inputs;
- incomplete inputs;
- contradictory inputs;
- noisy inputs;
- ambiguous requests;
- unsafe output force करने की कोशिशें;
- format edge cases।

fixture उदाहरण

```
{
  "id": "ticket-001",
  "input": "मुझे किसी से पिछले महीने की invoice चेक करवानी है क्योंकि मुझे लगता है कि वह गलत आई है।",
  "expected_traits": [
    "identifier missing हो तो clarification मांगें",
    "data invent न करें",
    "professional tone बनाए रखें",
    "next step propose करें"
  ]
}
```

अच्छा fixture किसे कहें

अच्छा fixture tricks से model को "हराने" की कोशिश नहीं करता। वह useful situation बताता है। वह होना चाहिए:

- stable;
- reproducible;
- readable;

- representative;
- easy to extend।

अगर fixture हर समय बदलता रहे तो version comparison नहीं हो सकता। अगर बहुत artificial हो तो real usage reflect नहीं करेगा। quality balance में है।

Golden tests: expected behavior को fix करना

golden tests expected output या बहुत specific output properties के against tests हैं। ये prompt, model, या tool chain बदलने पर regressions detect करते हैं।

दो सामान्य तरीके:

exact golden

जब output बहुत stable होना चाहिए, जैसे structured format।

उदाहरण:

```
{
  "id": "routing-01",
  "expected": {
    "category": "facturacion",
    "confidence": "alta"
  }
}
```

property-based golden

जब full text freeze नहीं करना चाहते, लेकिन behavior freeze करना चाहते हैं।

उदाहरण:

- response data invent न करे;
- warning शामिल हो;
- exactly 3 steps लौटाए;
- expected language का उपयोग करे;
- unauthorized tools mention न करे।

यह दूसरा format ज्यादा flexible है और natural language systems में अक्सर ज्यादा useful होता है। gold हमेशा exact string नहीं होता; कई बार वह observable rules का compliance होता है।

Harness CLI: चलाओ, compare करो, repeat करो

harness वह environment है जो fixtures के साथ prompts चलाता है, results log करता है, और उन्हें reference के खिलाफ compare करता है। एक अच्छी CLI इसे terminal और CI से repeatable बनाती है।

एक reasonable structure इन चीजों को अलग रखता है:

- prompt definition;
- fixture definition;
- model configuration;
- batch execution;
- evaluation;
- report;
- result export

संभावित structure

```
prompt-harness/  
  prompts/  
    system.md  
    task.md  
  fixtures/  
    inbox.jsonl  
    safety.jsonl  
    formatting.jsonl  
  evals/  
    rubric.md  
    scoring.ts  
  runs/  
    2026-05-08T10-30-00Z.jsonl  
  reports/  
    latest.md  
  src/  
    cli.ts  
    harness.ts  
    loader.ts  
    evaluator.ts
```

उदाहरण commands

```
node src/cli.js run --prompt prompts/system.md --fixtures fixtures/inbox.jsonl  
node src/cli.js eval --run runs/latest.jsonl --rubric evals/rubric.md  
node src/cli.js compare --baseline runs/baseline.jsonl --candidate runs/latest.jsonl  
node src/cli.js report --input runs/latest.jsonl --output reports/latest.md
```

CLI को क्या करना चाहिए

उपयोगी CLI सिर्फ model call नहीं करती। वह:

- files के अस्तित्व की validation करती है;

- inputs normalize करती है;
- prompt और model version log करती है;
- timestamps save करती है;
- raw responses serialize करती है;
- metrics compute करती है;
- CI के लिए clear summary देती है।

अगर harness सिर्फ सुंदर text print करे, तो वह demo है। अगर traceability और comparison भी दे, तभी वह infrastructure बनना शुरू करता है।

Evals और rubrics: "मुझे अच्छा लगा" से आगे

prompt evaluation सिर्फ intuition पर निर्भर नहीं हो सकता। आपको repeatable criteria चाहिए। rubric यही देता है: "good" का explicit definition।

simple rubric

rubric इन dimensions पर score दे सकता है:

- context fidelity;
- completeness;
- accuracy;
- format;
- tone;
- safety;
- actionable usefulness।

उदाहरण:

```
प्रति dimension score 0-2:  
0 = serious failure  
1 = partial  
2 = meets criteria
```

evaluation उदाहरण

Dimension	Criterion
Fidelity	data invent नहीं करता और input से contradict नहीं करता
Format	मांगी गई structure का पालन करता है
Tone	professional language बनाए रखता है
Actionability	useful next steps देता है
Safety	forbidden actions execute या recommend नहीं करता

automatic और human evaluation

दोनों approach का संयोजन बेहतर है।

- automatic: objective rules, format, length, field presence, prohibited patterns के लिए;
- human: quality nuance, clarity, persuasion, usefulness, alignment के लिए।

real systems में common error है weak automatic metrics को full solution मान लेना। वे useful हैं, पर critical reading का replacement नहीं। अच्छी प्रैक्टिस है filtering के लिए automation और decision के लिए human rubric।

Regression tests: जो पहले काम कर रहा था उसे टूटने मत दो

regression test versions के बीच behavior compare करता है। लक्ष्य system को हमेशा के लिए freeze करना नहीं, बल्कि unwanted changes detect करना है।

mature flow में हर prompt या model change को यह जवाब देना चाहिए:

1. क्या बेहतर हुआ;
2. क्या खराब हुआ;
3. कौन-से नए cases दिखे;
4. कौन-से trade-off स्वीकार हैं;
5. कहाँ rollback चाहिए।

typical regression cases

- नया prompt जरूरत से ज्यादा verbose हो जाता है;
- safety warning गायब हो जाती है;
- model clarification मांगना बंद कर देता है;
- language बदल जाती है;
- JSON format parser तोड़ देता है;
- tool तब invoke होता है जब नहीं होना चाहिए।

practical strategy

critical fixtures का छोटा set रखें और exploratory fixtures का बड़ा set रखें। critical set essentials बचाता है; exploratory set main path के बाहर behavior दिखाता है।

अगर regression test fail हो, तो सिर्फ "output ठीक करो" काफी नहीं। समझना होगा कि issue कहाँ है:

- prompt;
- model;
- post-processing;
- configuration;
- policy;
- data set।

Security और permissions

जब model actions produce करता है, सिर्फ अच्छा response काफी नहीं। उसे clear limits के भीतर operate भी करना होगा। tool-enabled systems में security prompt और harness design का हिस्सा है।

basic principles

- least privilege;
- sensitive actions के लिए confirmation;
- suggest और execute का separation;
- input और output validation;
- decision logging;
- dangerous actions का explicit blocking।

rule उदाहरण

अगर कोई action data modify करे, पैसे charge करे, जानकारी delete करे, या external messages भेजे, तो execute करने से पहले human confirmation लेना जरूरी है।

secure prompt

prompt को permissions या credentials assume नहीं करने चाहिए। उसे model को यह भी encourage नहीं करना चाहिए कि वह external validation वाली चीजें "अपने आप solve" कर ले।

उदाहरण:

यह assume मत करो कि तुम्हें external systems का access है। अगर impactful action चाहिए, तो उसे describe करो और confirmation मांगो।

secure harness

harness को permissions simulate करने और limits test करने में सक्षम होना चाहिए:

- बिना internet access;
- disabled tools के साथ;
- fake credentials के साथ;
- read-only mode में;
- confirmation required mode में।

इससे verify होता है कि system सिर्फ full-access mode में नहीं, बल्कि restricted conditions में भी सही चलता है।

Observability: सच में क्या हुआ, यह देख पाना

observability वह क्षमता है जो system के lab से बाहर जाने के बाद debugging और learning संभव बनाती है। अगर production में prompt fail होता है, तो context reconstruct करना जरूरी होता है।

क्या log करना चाहिए

- prompt version;
- model version;
- date और time;
- summarized input;
- raw output;
- tools used;
- latency;
- estimated cost;
- errors;
- policy decision;
- fixture या case identifier।

log उदाहरण

```
{
  "run_id": "2026-05-08T10:30:00Z",
  "prompt_version": "1.4.2",
  "model": "gpt-5.3",
  "fixture_id": "safety-03",
  "latency_ms": 1840,
  "tools_used": ["search"],
  "outcome": "needs_review"
}
```

पहले क्या देखना चाहिए

जब कुछ fail हो, तो यह क्रम उपयोगी है:

1. original input;
2. actual applied prompt;
3. model configuration;
4. available tools;
5. raw output;
6. post-processing;
7. evaluation criteria

observability के बिना हर bug जादू जैसा लगता है। observability के साथ वह auditable decisions की sequence बन जाता है।

एक minimal harness उदाहरण

idea को जमीन पर लाने के लिए मान लें कि system internal requests classify करता है और short plan देता है। minimum architecture कुछ ऐसा हो सकता है:

1. persistent instructions load करो
2. task prompt load करो
3. fixture load करो
4. final message बनाओ
5. model चलाओ
6. format validate करो
7. rubric से score करो
8. result save करो
9. baseline से compare करो
10. report करो

pseudocode

```
const system = loadFile("prompts/system.md")
const task = loadFile("prompts/task.md")
const fixtures = loadJsonl("fixtures/inbox.jsonl")

for (const fixture of fixtures) {
  const input = buildInput(system, task, fixture)
  const output = await model.run(input)
  const score = evaluate(output, fixture.expected_traits)
  saveRun({ fixtureId: fixture.id, output, score })
}
```

इस उदाहरण का मुख्य बिंदु

शुरुआत के लिए sophistication जरूरी नहीं। जरूरी यह है कि flow:

- explicit हो;
- reproducible हो;
- versioned हो;
- evaluable हो;
- comparable हो।

यह मौजूद हो तो scale करना संभव है। इसके बिना maintenance मुश्किल है।

prompts को iterate करने के criteria (system तोड़े बिना)

prompt सुधारते समय blind edits मत करो। एक short और disciplined cycle बेहतर है।

iteration checklist

- exact problem define करो;
- representative fixture चुनो;
- improvement hypothesis लिखो;
- एक बार में एक ही चीज बदलो;
- harness चलाओ;
- baseline से compare करो;
- नए failures review करो;
- change accept करो या revert करो;
- decision document करो।

practical rules

1. format improvements और policy changes को same iteration में mix मत करो।
2. सिर्फ एक happy example से change justify मत करो।
3. regressions देखे बिना victory declare मत करो।
4. version और rationale के बिना नया prompt save मत करो।
5. critical business logic को सिर्फ prompt के अंदर मत रखो, अगर वह code में बेहतर रह सकती है।

कब logic को prompt से बाहर ले जाना चाहिए

prompt को सब कुछ ढोना नहीं चाहिए। अगर कोई rule strict, verifiable, और business-central है, तो अक्सर उसे model के बाहर code करना बेहतर है।

उदाहरण:

- schema validation;
- permission rules;

- deterministic routing;
- sensitive data filtering;
- format normalization;
- metric computation।

prompt वही करे जिसमें वह अच्छा है: interpret करना, prioritize करना, लिखना, synthesize करना, और context के साथ निर्णय लेना। code वही करे जिसमें वह अच्छा है: validate करना, control करना, और deterministic execution।

maturity signals

आप "loose prompt" से "system" में तब जाते हैं जब इन सवालों का जवाब बिना improvisation दे सकें:

- production में कौन-सा prompt version है?
- किन fixtures से validate होता है?
- rubric कौन-से criteria define करता है?
- baseline के मुकाबले क्या बदला?
- कौन-से actions allowed हैं?
- logs में क्या रिकॉर्ड हुआ?
- कौन-सी regressions tolerate हैं और कौन-सी नहीं?

अगर ये जवाब बिखरे हुए हों, तो system अभी भी human memory पर ज्यादा निर्भर है।

operational summary

prompt engineering सिर्फ बेहतर लिखना नहीं है। यह ऐसी instruction interface बनाना है जो समय के साथ टिक सके। harness engineering वही support है जो इस sustainability को संभव बनाता है: fixtures load करना, versions run करना, results evaluate करना, evidence record करना, और accidental changes से system को बचाना।

practical discipline एक simple formula में समेटी जा सकती है:

- persistent instructions और task prompts को अलग करो;
- real और representative fixtures उपयोग करो;
- golden tests और rubrics से expectations fix करो;
- changes publish करने से पहले regression tests चलाओ;
- permissions और sensitive actions control करो;
- production behavior observe करो;
- जो भी महत्वपूर्ण है उसका version रखो।

जब यह सब मौजूद होता है, prompt एक bet नहीं रहता। वह एक engineering component बन जाता है।

final checklist

- ☐ system की persistent instruction define करो।

- ☐ task prompt को dynamic context से अलग करो।
- ☐ happy, ambiguous, और risky cases के लिए fixtures बनाओ।
- ☐ हर critical behavior के लिए कम से कम एक golden test लिखो।
- ☐ simple और repeatable rubric design करो।
- ☐ `run` , `eval` , और `report` के साथ harness CLI implement करो।
- ☐ prompts, model, और configuration का versioning save करो।
- ☐ हर execution की logs और latency record करो।
- ☐ permissions, fallbacks, और no-tool cases test करो।
- ☐ हर change को baseline के against compare करो।
- ☐ change और rollback decisions document करो।

इतना सेटअप एक promising idea को productive, auditable, और maintainable capability में बदलने के लिए पर्याप्त है।